Listing 6.9 Integrating an agent into the conversation chain

```
const tools = [new WikipediaQueryRun({
  topKResults: 3,
  maxDocContentLength: 4000,
  handleValidationError: (error) =>
console.error('Search validation error:', error)
})];

const AGENT_SYSTEM_TEMPLATE = `You are a helpful AI assistant
specializing in technical queries and web technologies.
When using WikipediaQueryRun for searches:
Prioritize technical documentation and standards
Cross-reference information from multiple sources
Format code examples using markdown
Example interaction: User: Explain WordPress webhook architecture
Action: WikipediaQueryRun(search="WordPress webhook system")
Response: WordPress webhooks use... [technical details]`;

const prompt = ChatPromptTemplate.fromMessages([
  ['system', AGENT_SYSTEM_TEMPLATE],
  ['human', '{input}'],
  new MessagesPlaceholder('agent_scratchpad'),
]);

const agent = createReactAgent({
  llm: model,
  tools,
  prompt,
});
const output = await agent.invoke({
  messages: [new HumanMessage("Everest")],
});
```

Initializes an array of tools with a WikipediaSearch instance that limits results to a maximum of the top three results

Defines a template string for the agent's system message

Creates a new agent instance using the specified language model, tools, and prompt

Invokes the agent with a specific input message and awaits its response

An agent in LangChain combines three key parts: a system prompt that defines behavior, a set of tools for taking actions, and a language model for reasoning. The `createReactAgent` function implements the ReAct framework, where the model not only generates answers but also decides when to use tools like Wikipedia search. This design matters because it transforms a static model into a dynamic problem solver—able to reason step by step, call external tools, and incorporate fresh, relevant information. Currently, ReAct is the main agent type supported in LangChain, but the framework may expand with new approaches in the future.

Files: agents.google.js, agents.google.js

The code for this example project is in the ch06 folder, including the code creating agents in LangChain. You can use that example by running this command in the source folder if using OpenAI :

```
$ node agents.openai.js
```